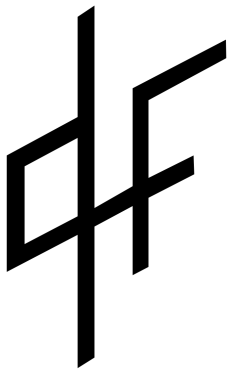
RAPPORT DE SOUTENANCE DU PROJET OCR

par la team qlf



Alexandre Pern
Eliot Soumeillan
Jean-Philippe Vonthron
Mathis Lafage

Sommaire

I. Présentation des membres..... 3
a. Alexandre Pern..... 3
b. Eliot Soumeillan..... 3
c. Jean-Philippe Vonthron..... 3
d. Mathis Lafage..... 3
II. Planification et répartition des tâches..... 4
III. État d'avancement du projet à la première soutenance.....5
a. Le réseau de neurones XNOR..... 5
c. Rotation de l'image manuelle.....7
e. Interface utilisateur.....11
f. Solver.....12
IV. État d'avancement du projet à la dernière soutenance..... 14
a. Architecture et Implémentation du Réseau de Neurones..... 14
1. Architecture du Modèle..... 14
2. Propagation Avant (Forward Propagation)..... 14
3. Apprentissage et Optimisation..... 15
4. Régularisation et Robustesse..... 16
b. Adaptation et amélioration du Solver..... 17
1. Recherche Approximative (Fuzzy Matching)..... 17
c. Rotation automatique de l'image..... 18
e. Interface (GUI)..... 22
f. Pipeline de Traitement Global (Main OCR)..... 26
1. Prétraitement et Normalisation..... 26
2. Segmentation.....27
3. Reconnaissance Optique.....27
4. Résolution et Visualisation.....27
g. Segmentation, Extraction et Filtrage des Caractères..... 29
1. Introduction..... 29
2. Segmentation de l'Image..... 29
2.a – Détection des Composantes Connexes (BFS)..... 29
2.b – Clustering via DBSCAN..... 29
2.c – Classification des Zones..... 30
3. Extraction des Lettres..... 32
3.a – Extraction Initiale et Assignation..... 33

3.b – Séparation des Caractères Fusionnés.....33
3.c – Filtrage Avancé..... 35
4. Ajout du Bouton “Filtre Médian”..... 35
4.a – Interface Utilisateur..... 35
4.b – Traitement dans le Programme.....35
V. Conclusion..... 39
VI. Bibliographie..... 42

I. Présentation des membres

a. Alexandre Pern

Je m'appelle Alexandre Pern et je suis un membre du groupe QLF pour le projet OCR. Je suis chargé de développer le **réseau de neurones** permettant la reconnaissance intelligente des caractères de l'image d'entrée, le **solveur** de grille et que la pipeline de l'ocr.

b. Eliot Soumeillan

Je me nomme Eliot Soumeillan. Dans le cadre du projet OCR du groupe QLF, j'ai pris en charge le développement des modules dédiés à la **détection** de la grille ainsi qu'à la détection et à la reconnaissance des mots et des lettres sur des images.

c. Jean-Philippe Vonthron

Je m'appelle Jean-Philippe Vonthron, je suis un membre du groupe QLF pour le projet OCR. Mon rôle dans ce projet est le développement de la **rotation** d'image ainsi que de l'**interface utilisateur**.

d. Mathis Lafage

Je m'appelle Mathis Lafage et je fais partie du groupe de projet QLF. Durant ce semestre, je suis chargé du **prétraitement** de l'image. Il s'agit d'une tâche cruciale pour le bon fonctionnement de l'OCR.

II. Planification et répartition des tâches

Tout d'abord, nous avons séparé le projet en plusieurs tâches distinctes. Le choix de ces tâches a été très judicieux car il nous a permis de faire le projet en le divisant en plusieurs sous-programmes, où chaque membre du groupe est dédié à un seul sous-programme. Une fois que tous les sous-programmes seront achevés, nous pourrons développer un programme principal qui mettra tout bout à bout et nous aurons ainsi notre solveur de mots cachés fonctionnel. Concernant la planification de toutes ces tâches, nous nous sommes tout d'abord référé aux prérequis pour la première soutenance, puis nous avons évalué la difficulté et la priorité de chaque tâche, pour déterminer par quoi commencer et où aboutir.

Tâches	Alexandre	Eliot	Jean-Philippe	Mathis
	Imagerie			
Prétraitement de l'image				X
Rotation de l'image			X	
Détection de la grille et de la liste de mots		X		
Détection des lettres et des mots		X		
	Réseau de neurone			
Réseau de neurones	X			
	Autres			
Interface Utilisateur			X	
Solveur	X			

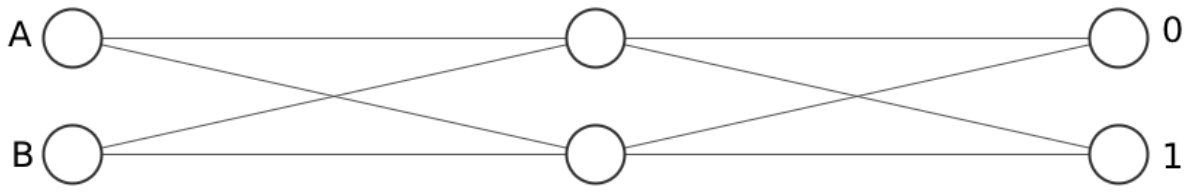
Nous avons décidé de commencer par le prétraitement de l'image car c'est un prérequis pour d'autres tâches et c'est aussi le traitement au tout début de la *pipeline OCR*. En effet, pour découper l'image, nous sommes obligés de travailler sur une image binarisée, sans bruit etc. Nous avons bien avancé sur le réseau de neurones car c'est le cœur du projet et car il s'agit d'une notion complexe à comprendre et à implémenter. De plus, le découpage de l'image a aussi été une priorité durant le développement et la conception du projet car nous considérons que l'une des tâches les plus complexes et qu'elle nécessite plus de temps que d'autres.

III. État d'avancement du projet à la première soutenance

Concernant l'avancement du projet, nous considérons ne pas avoir de retard sur les critères attendus pour la deuxième soutenance. Nous avons même pris de l'avance sur le réseau de neurones en ayant bien avancé le réseau principal reconnaissant les lettres.

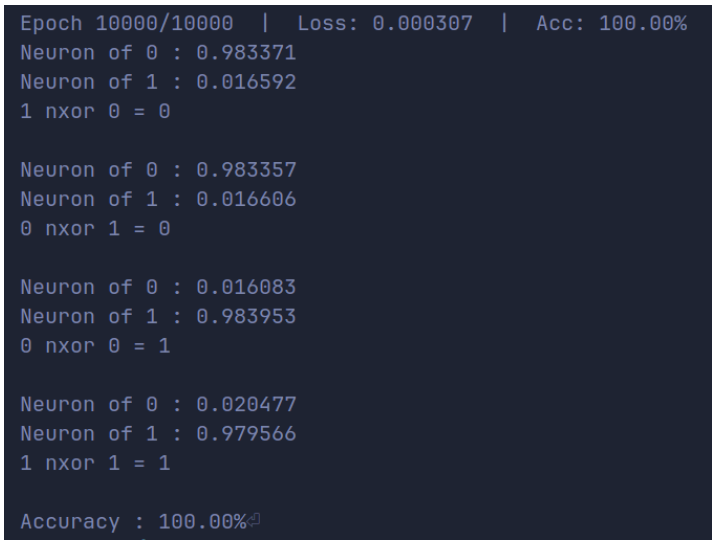
a. Le réseau de neurones XNOR

Il était demandé de faire une preuve de concept d'un réseau de neurones en réalisant un réseau capable d'apprendre la **fonction logique XNOR**. Nous avons alors réalisé un mini-réseau de neurones contenant 3 couches : une couche d'entrée, une couche cachée et une couche de sortie. Pour rendre le réseau uniforme, nous avons mis 2 neurones pour la couche cachée et pour avoir ainsi 2 neurones sur chaque couche. Nous pouvons donc représenter le réseau de neurones comme ceci :



Pour entraîner notre réseau de neurones à résoudre la fonction logique XNOR, nous avons opté pour une fonction d'activation sigmoïde sur les couches, afin d'obtenir des valeurs comprises entre 0 et 1. Afin d'ajuster les poids du réseau, nous avons

utilisé la **descente de gradient** couplée à la **rétropropagation**. Concrètement, nous avons d'abord défini une fonction de perte quadratique (MSE) qui mesure l'écart entre les prédictions du réseau et les sorties attendues. Ensuite, lors de la rétropropagation, nous avons calculé les gradients de cette perte par rapport à chaque poids en appliquant la **règle de la chaîne**, ce qui nous a permis de propager l'erreur depuis la sortie vers les couches précédentes. Enfin, nous avons mis à jour les poids dans le sens opposé au gradient (avec un taux d'apprentissage de 0,5) pour minimiser progressivement l'erreur.
Nous aurions pu faire une couche de sortie avec une seule neurone et seulement regarder la valeur de ce neurone mais nous avons préféré faire deux neurones pour chaque sortie. En effet, ce modèle est réutilisable pour la reconnaissance des lettres, où chaque neurone de sortie correspond à une lettre de l'alphabet.



Voici un test de la table de vérité de l'opération logique XNOR après un entraînement du réseau. On peut observer une activation notable du neurone correspondant au bon résultat de chaque opération.

b. Prétraitement de l'image

Lorsqu'on prétraite une image pour un système de reconnaissance visuelle, notre objectif est simple : nettoyer et simplifier l'image pour en extraire l'essentiel. D'abord, nous convertissons l'image en **niveaux de gris**, ce qui permet de réduire les informations superflues tout en conservant les contrastes nécessaires à l'analyse. Ensuite, nous appliquons un **filtre médian** qui élimine les imperfections, comme les pixels isolés ou les parasites, sans altérer les contours importants. Puis, grâce à une méthode de **seuillage Otsu**, nous déterminons le meilleur seuil pour séparer clairement le noir du blanc. Enfin, nous **binarisons** l'image pour obtenir une version noire et

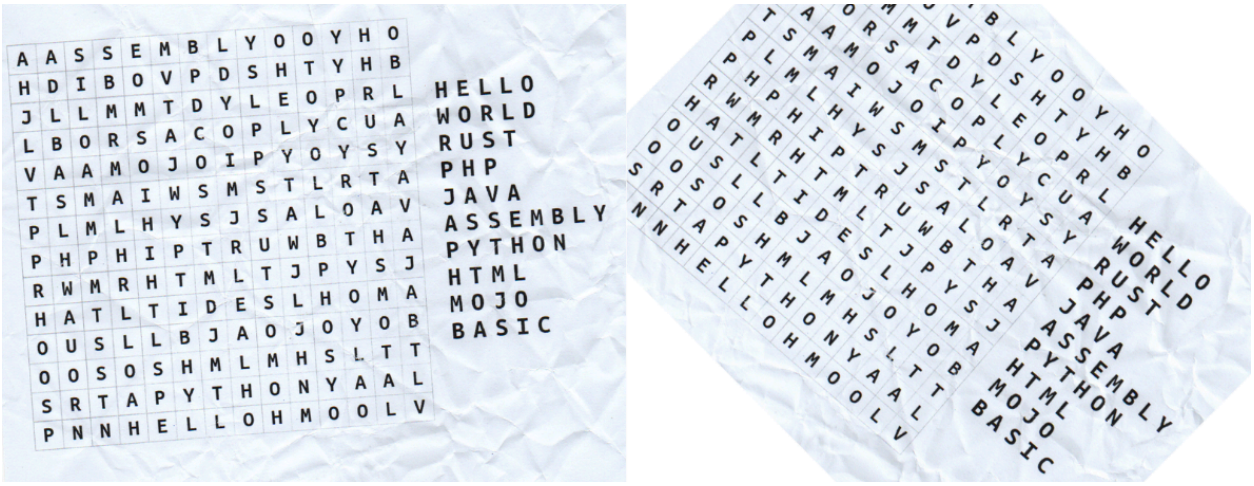
blanche, idéale pour une reconnaissance précise. Ce processus garantit que l'image finale soit propre, contrastée et prête à être analysée par nos algorithmes.





c. Rotation de l'image manuelle

Lorsqu'on applique une **rotation** à une image dans le cadre d'un traitement visuel, notre objectif est d'ajuster son orientation sans altérer sa qualité ni perdre d'informations importantes. Tout d'abord, nous calculons l'angle de rotation souhaité, puis déterminons la position de chaque pixel dans la nouvelle image à partir de sa position d'origine. Pour garantir une transformation fluide et conserver la précision des détails, nous utilisons une **interpolation bilinéaire** : cette méthode estime la couleur d'un pixel en combinant les valeurs de ses voisins les plus proches. Ainsi, les transitions restent douces et les contours nets, même après rotation. Une fois la transformation terminée, l'image pivotée est sauvegardée sous la forme d'un nouveau fichier de sortie, assurant la conservation du résultat tout en laissant intacte l'image d'origine. Cette étape finale permet de disposer d'une version corrigée, proprement orientée et prête à être utilisée pour l'analyse.



Problèmes rencontrés:

Au cours du développement de la fonction de rotation d'image, plusieurs difficultés techniques ont été rencontrées et progressivement résolues. Le premier problème concernait la dépendance à la bibliothèque *libpng*, initialement utilisée pour manipuler les images. Comme le projet devait fonctionner sans cette dépendance car non disponible sur les pc de l'école, la solution retenue a été d'utiliser la bibliothèque GdkPixbuf, intégrée à GTK, qui permet de charger, modifier et sauvegarder des images de manière plus légère et directe.

Ensuite, les premières versions de la rotation entraînaient une perte de pixels, notamment au niveau des bords de l'image. Ce phénomène provient du fait que la rotation s'effectuait à l'intérieur du cadre d'origine, ce qui coupait les coins après transformation. Pour corriger cela, une première approche a consisté à agrandir la surface de destination afin de contenir l'ensemble de l'image tournée. Cependant, cette solution provoquait un autre effet indésirable : la taille de l'image variait après rotation, ce qui n'était pas acceptable pour l'application.

Une version ultérieure a donc été modifiée pour conserver la taille originale du canevas tout en remplissant les zones vides avec un fond blanc. Toutefois, cette méthode a provoqué de nouveaux problèmes, tels que des "trous" ou des effets d'escalier. Ces imperfections provenaient de l'utilisation d'un simple arrondi lors du calcul des coordonnées des pixels. Pour obtenir un rendu plus concluant, une interpolation bilinéaire a été implémentée. Cette technique combine les couleurs des quatre pixels les plus proches afin de calculer la valeur du pixel tourné, supprimant ainsi les pertes visuelles.

Un autre problème est apparu lorsque l'utilisateur effectue plusieurs rotations successives : l'image se dégradait progressivement. Cette perte de qualité provenait du fait que chaque rotation était appliquée sur le résultat déjà tourné, accumulant les

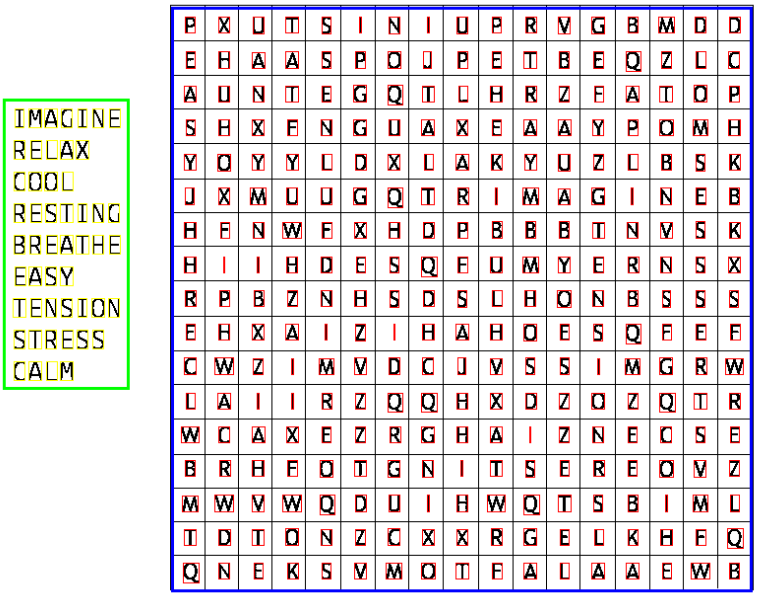
erreurs d'arrondi et les interpolations. La solution a consisté à conserver une copie de l'image d'origine, de sorte que chaque rotation soit recalculée directement à partir de cette image source, garantissant ainsi une qualité constante.

De plus, il a fallu stabiliser le centre de rotation, qui pouvait légèrement se décaler entre deux transformations. Le calcul du pivot a donc été fixé sur le centre de l'image originale, permettant une rotation centrée et une taille inchangée.

Ainsi, à travers ces différentes étapes, la fonction de rotation a été progressivement améliorée jusqu'à obtenir un résultat stable, précis et visuellement sans perte.

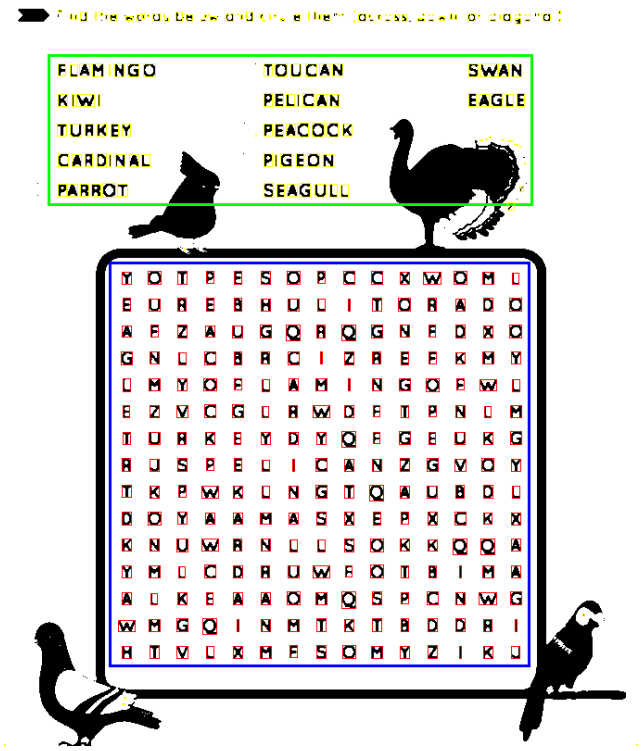
d. Découpage de l'image

Pour clarifier la distinction entre la zone "grille" et la zone "liste de mots", nous partons d'une lecture globale de la page puis affinons par contrastes de régularité. D'abord, nous analysons la structure spatiale: la grille se manifeste par un maillage très régulier de lignes et de colonnes, avec des espacements quasi constants; la liste, à l'inverse, présente des colonnes plus lâches, des lignes parfois incomplètes et des variations d'espacement. Cette différence de "rythme" géométrique guide toute la séparation.



Concrètement, nous regroupons les lettres selon leur proximité naturelle pour faire émerger des ensembles compacts, puis nous qualifions ces ensembles par des

métriques de régularité: stabilité des lignes (espacements verticaux homogènes), présence de colonnes cohérentes (pics stables dans l'axe horizontal), compacité et densité interne. Un ensemble marquant des lignes et colonnes nettes, avec une variance d'espacement faible, sera identifié comme "grille"; un ensemble au contraire moins dense, aux colonnes irrégulières et aux espacements plus variables, sera identifié comme "liste". Cette lecture se fait à l'échelle locale (à l'intérieur de chaque zone) et à l'échelle globale (comment ces zones s'agencent entre elles). Pour réaliser cela nous avons utilisé la méthode **K-means** mais celle-ci n'était pas tout à fait adaptée à notre situation donc nous nous sommes penchés sur la méthode **DBscan** qui nous a apporté des solutions.

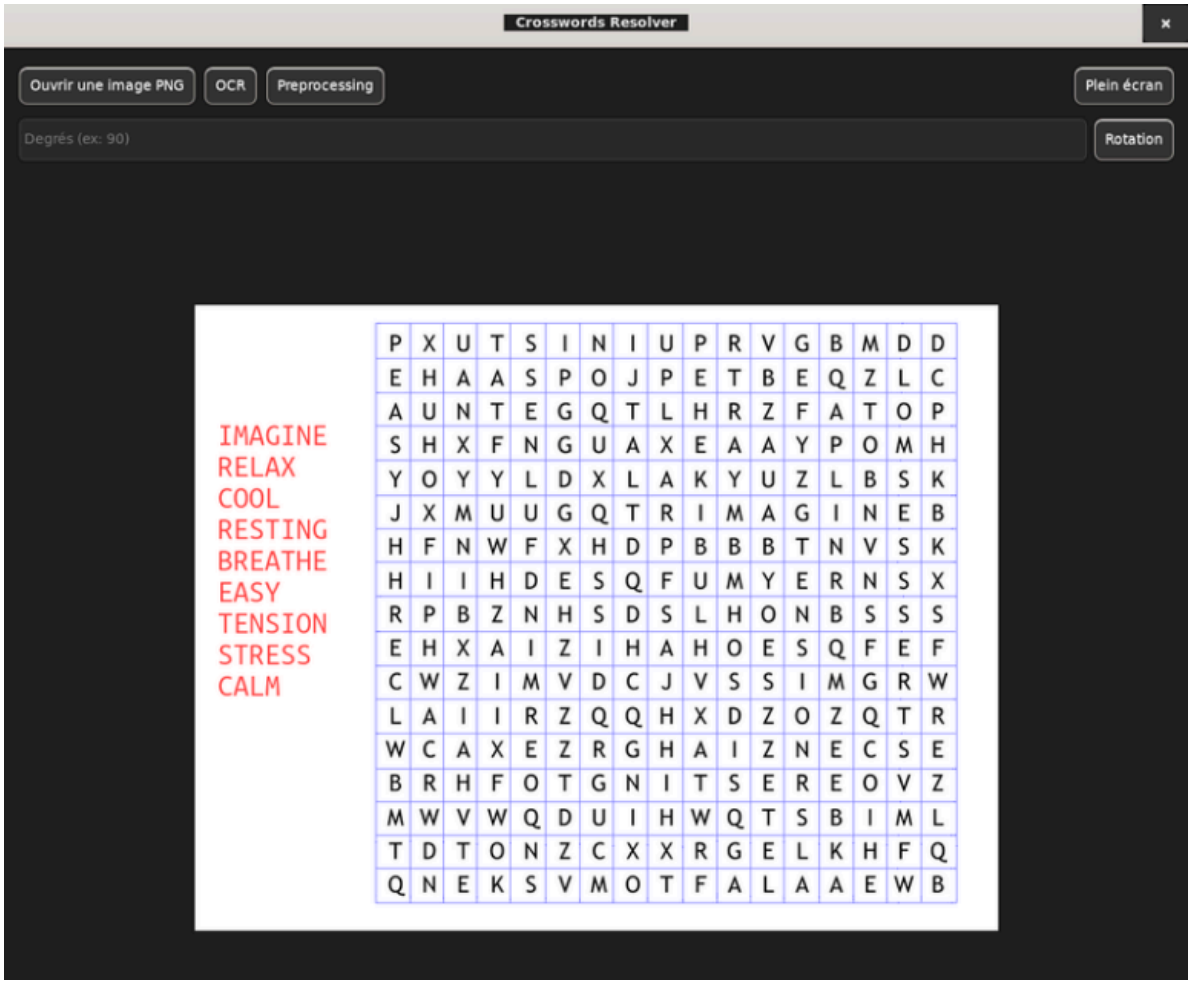


Afin de préserver les lettres fines et mieux gérer les cas limites, nous avons adapté le filtre de préparation d'image pour qu'il distingue plus efficacement les lettres des autres éléments graphiques (traits décoratifs, bordures, illustrations). Cette adaptation vise à conserver les détails utiles tout en réduisant les artefacts qui perturbent la lecture. Après séparation, un léger post-traitement typographique consolide les unités de lecture: recoller des "l" ou des traits interrompus, séparer des tirets longs fusionnés, harmoniser l'alignement vertical. Dans les rares situations ambiguës — par exemple une liste très bien alignée ou une grille légèrement déformée — un arbitrage final s'appuie sur un petit ensemble de descripteurs de régularité, de

densité et d'alignement pour confirmer l'étiquette "grille" ou "liste". Cette démarche, centrée sur la régularité géométrique plutôt que sur des seuils fixes, offre une séparation visuelle plus robuste, lisible et fidèle à la structure réelle du document. Le filtre reste toutefois perfectible et fera l'objet d'affinements supplémentaires pour améliorer encore la distinction lettres/éléments non textuels.

e. Interface utilisateur

L'interface graphique a été conçue avec GTK 3 afin d'offrir une utilisation simple et intuitive du programme. Elle permet de charger une image, d'appliquer une rotation selon un angle défini par l'utilisateur et d'afficher instantanément le résultat. L'interface comprend plusieurs boutons : Ouvrir une image, Rotation, Preprocessing, OCR et Plein écran, regroupés dans une disposition claire. Un champ de saisie permet d'entrer la valeur de l'angle de rotation, tandis qu'une zone centrale affiche l'image avant et après transformation. Un thème sombre a été appliqué pour améliorer le confort visuel et donner une apparence moderne à l'application. La rotation est effectuée directement au sein de l'interface, à l'aide du programme externe de rotation mentionné précédemment. L'image transformée est également enregistrée automatiquement sous le nom "output.bmp". Cette interface rend ainsi le traitement d'images plus accessible et visuel. Cependant les boutons OCR et Preprocessing ne sont pour l'instant pas liés à leur programme donc sont hors service pour l'instant.



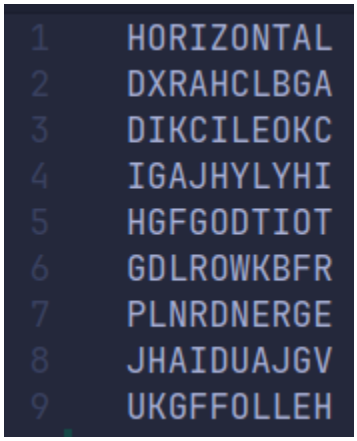
Problème rencontré: Le bouton Plein écran rencontre des problèmes avec l'image de test "test_2_image_2" à cause de son format, problème qui n'est pas encore fixé pour le moment.

f. Solver

Il nous était demandé de réaliser un **solveur en ligne de commande** qui renvoie les coordonnées de départ et de fin dans une grille d'un mot passé en paramètre. Pour cela, nous avons d'abord mis chaque lettre du fichier texte passé en paramètre dans une matrice de deux dimensions. Ensuite, nous parcourons cette matrice jusqu'à trouver la première lettre du mot, puis nous regardons dans les huit directions (droite,

gauche, diagonales etc.) si le mot est présent. Si nous trouvons le mot, nous renvoyons les coordonnées de départ et de fin du mot.





IV. État d’avancement du projet à la dernière soutenance.

a. Architecture et Implémentation du Réseau de Neurones

Le coeur du système de reconnaissance de caractères optique (OCR) repose sur un réseau de neurones artificiels de type **Perceptron Multicouche (MLP)**.

1. Architecture du Modèle

Le modèle est conçu pour classer des images de caractères de **28×28 pixels** en **26 classes** correspondant aux lettres de l'alphabet (A-Z).
Il se compose de **4 couches**, structurées comme suit :

- **Couche d'entrée (Input)** :
  784 neurones (28 × 28 pixels), recevant les valeurs de gris normalisées de l'image.

- **Couches cachées (Hidden Layers)** :

  - Une première couche de **128 neurones**.
  - Une seconde couche de **128 neurones**.
    Cette profondeur permet au réseau de capturer des caractéristiques abstraites de plus en plus complexes (courbes, boucles, intersections).
- **Couche de sortie (Output)** :
  26 neurones, représentant la probabilité d'appartenance à chaque classe (lettre).

2. Propagation Avant (Forward Propagation)

Les données circulent de l'entrée vers la sortie en subissant des transformations linéaires et non-linéaires :

- **Combinaison Linéaire** :
  Chaque neurone calcule une somme pondérée des activations de la couche précédente, ajustée par un biais
  $z=Wx+b$.

- **Fonction d'Activation (ReLU)** :
  Les couches cachées utilisent la fonction ReLU (Rectified Linear Unit), définie par
  $f(x)=\max(0,x)$.
  Elle permet d'introduire la non-linéarité nécessaire pour résoudre des problèmes complexes tout en évitant le problème de disparition du gradient (*vanishing gradient*) commun avec les sigmoïdes.

- **Fonction de Sortie (Softmax)** :
  La couche finale applique la fonction **Softmax**, qui transforme les scores bruts en une distribution de probabilité dont la somme vaut 1.
  Cela permet d'interpréter la sortie comme un niveau de confiance pour chaque lettre.

3. Apprentissage et Optimisation

L'entraînement du réseau est supervisé et repose sur l'algorithme de **rétropropagation du gradient (Backpropagation)** :

- **Fonction de Coût (Loss)** :
  Utilisation de l'**Entropie Croisée (Cross-Entropy Loss)**, particulièrement adaptée aux problèmes de classification multi-classes, pour quantifier l'écart entre la prédiction et la classe réelle.

- **Initialisation des Poids** :
  Pour assurer une convergence rapide et stable, les poids sont initialisés via la méthode de **Xavier (ou Glorot)**, qui ajuste la variance des poids en fonction du nombre de connexions entrantes et sortantes.

- **Optimiseur** :
  La **descente de gradient** est améliorée par l'ajout d'un terme de **Momentum (0.9)**.
  Le Momentum permet d'accélérer la convergence en accumulant la vitesse des gradients précédents, réduisant ainsi les oscillations dans les directions de forte courbure.

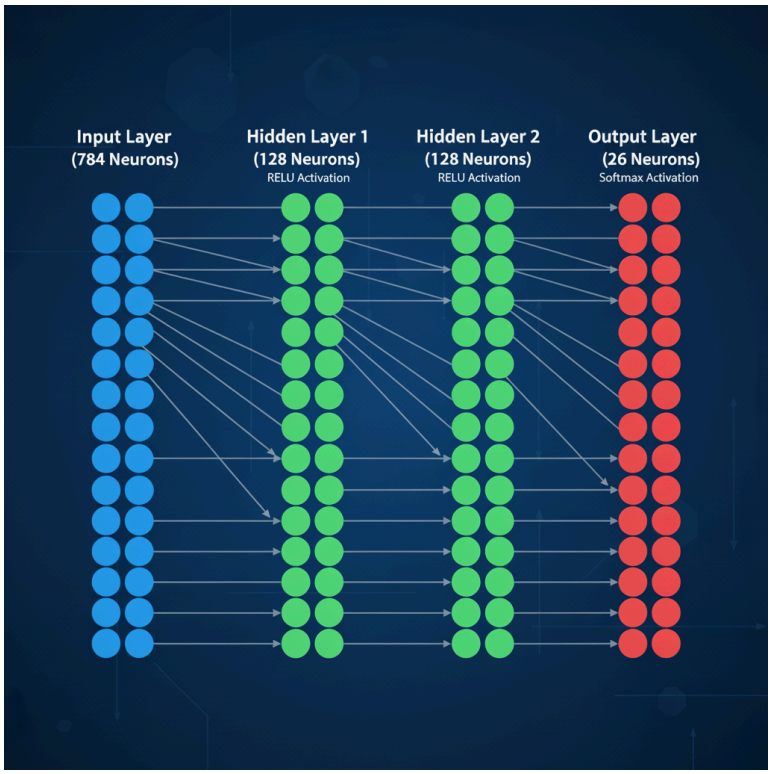
- **Taux d'Apprentissage (Learning Rate)** :
  Un taux d'apprentissage adaptatif est utilisé, qui diminue progressivement au fil des époques :

Cela permet de faire de grands pas au début et d'affiner les poids vers la fin.

4. Régularisation et Robustesse

Pour éviter le sur-apprentissage (overfitting) et assurer que le modèle généralise bien sur des données inconnues :

- Les données d'entraînement sont **mélangées (shuffled)** aléatoirement à chaque époque pour briser les corrélations temporelles.
- Le code intègre une structure pour la **régularisation L2** (pénalisation des grands poids), bien que le coefficient *Lambda* soit ajusté dynamiquement selon les besoins de l'entraînement.



Graphique arbitraire du réseau de neurones utilisé.

Le réseau de neurones a beaucoup évolué durant la conception du projet. En effet, nous avons rencontré plusieurs problèmes avant qu'il soit fonctionnel sur un large éventail de données. Au début, il s'agissait d'un réseau simple avec une couche cachée et une fonction d'activation sigmoïde sur toutes les couches. Nous l'avons par la suite peaufiné en ajoutant plusieurs détails comme le Momentum, la ReLU etc. Aujourd'hui

nous avons un modèle adapté qui s’entraîne rapidement et qui est fonctionnel sur les grilles d’exemple.

b. Adaptation et amélioration du Solver

Nous avons amélioré le module Solver pour qu’il soit robuste face aux erreurs de reconnaissance optique (OCR) grâce à une logique de comparaison approximative (*fuzzy matching*).

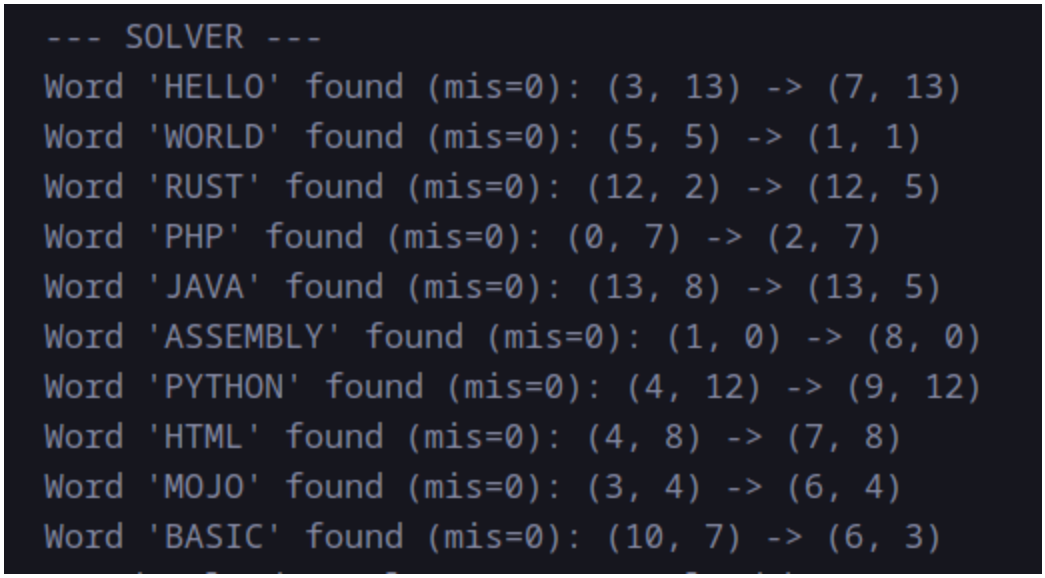
1. Recherche Approximative (Fuzzy Matching)

Une particularité clé de ce solveur est sa capacité à tolérer des erreurs. L'OCR n'étant pas infaillible, le mot recherché peut ne pas correspondre exactement aux caractères présents dans la grille.

- **Distance de Hamming :**
  L'algorithme compare, caractère par caractère, le mot cible avec la séquence présente dans la grille sur une direction donnée.

- **Seuil de Tolérance :**
  Un paramètre **max_mismatches** définit le nombre maximal de caractères incorrects tolérés.
  Si le nombre de différences reste inférieur ou égal à ce seuil, la séquence est considérée comme valide.

- **Sélection du Meilleur Candidat :**
  La fonction **find_best_match** ne s'arrête pas à la première occurrence trouvée. Elle parcourt l'intégralité de la grille pour identifier l'occurrence **minimisant le nombre d'erreurs**, garantissant ainsi de trouver le résultat le plus probable statistiquement.



Sortie du solver sur une grille d'exemple

Nous avons rencontré plusieurs problèmes avec le solver et nous l'avons modifié au fil du temps. Tout d'abord, quand nous avons ajouté la marge d'erreur, le solver trouvait des solutions erronées. En effet, si il existe une suite de lettre qui ressemble beaucoup au mot elle va le choisir même si le mot existe après dans le parcours de la grille. Pour cela, nous avons fait la fonction **find_best_match** décrite précédemment.

c. Rotation automatique de l'image

La rotation automatique des images a été conçue afin de corriger l'inclinaison sans intervention de l'utilisateur. Cette fonctionnalité repose sur une analyse du contenu de l'image permettant d'estimer l'angle de désalignement. Dans un premier temps, l'image est convertie en niveaux de gris grâce au prétraitement. Il permet de distinguer les zones sombres, généralement représentatives du texte ou des éléments importants, du fond de l'image. Cette simplification facilite l'analyse de l'orientation globale.

Ensuite, une recherche d'angle est effectuée sur un intervalle compris entre -45° et +45°, avec un pas régulier. Pour chaque angle testé, une projection horizontale est calculée en comptabilisant le nombre de pixels sombres par ligne après une rotation virtuelle de l'image. La variance de cette projection est alors mesurée. L'angle produisant la variance la plus élevée est considéré comme l'angle optimal, car il

correspond à un alignement maximal des structures horizontales présentes dans l'image, telles que les lignes de texte.

Une fois l'angle détecté, la rotation finale est appliquée à l'image d'origine. Cette rotation utilise une interpolation bilinéaire afin de garantir une meilleure qualité visuelle et d'éviter les effets de pixellisation. Les valeurs des pixels sont recalculées à partir de leurs voisins, ce qui permet d'obtenir une image plus lisse et plus fidèle à l'original.

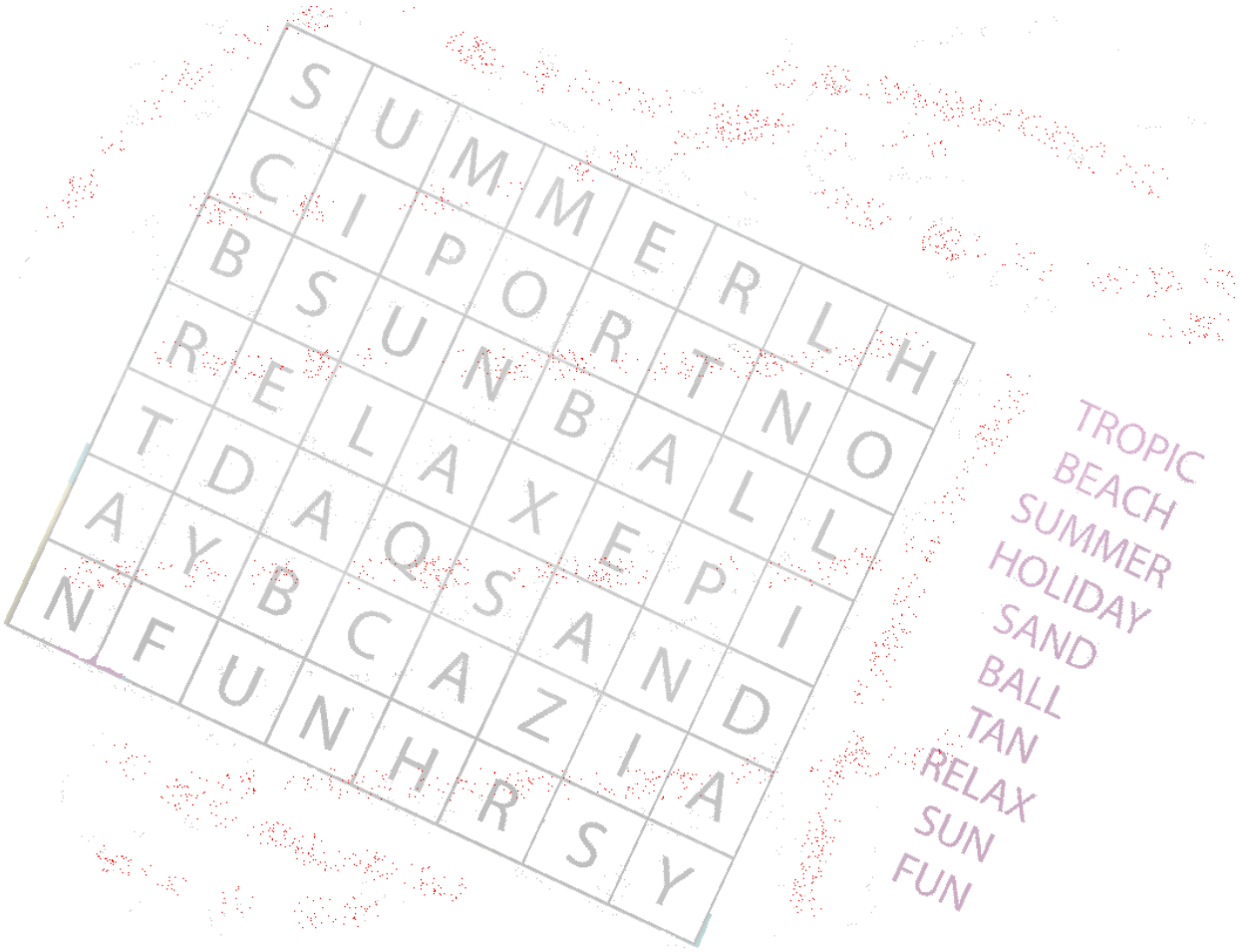
Ce module de rotation automatique est intégré comme une étape de prétraitement dans le projet global. Son rôle est d'améliorer la qualité des images avant l'exécution des traitements suivants. Il peut être activé ou désactivé selon les besoins et réutilisé dans différents contextes du projet, contribuant ainsi à une meilleure robustesse et à une plus grande flexibilité de l'application.

```
agent@JPAV:~/SPE/PROJET/QLF-OCR/RotationPNG$ ./rotation level 2 image 2.png auto test.bmp
```

Ligne de commande à entrer si on veut l'exécuter depuis un terminal
Concernant la rotation nous avons rencontré différents problèmes tels que:

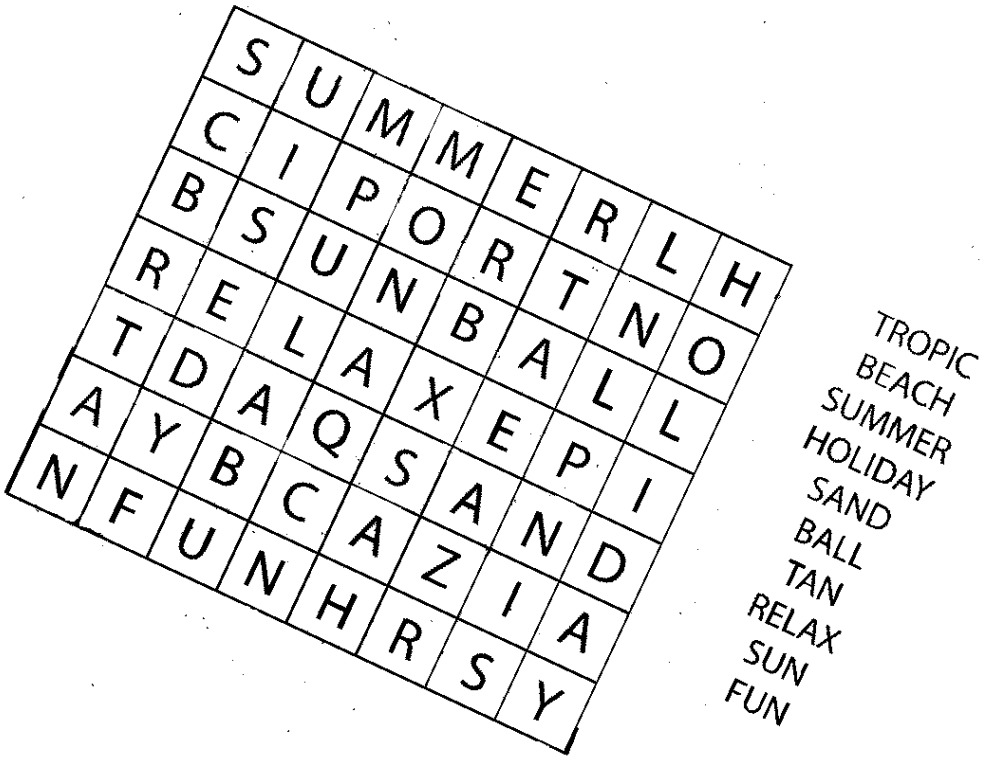
Le premier problème concerne la **détection de l'angle d'inclinaison**, qui repose sur l'analyse des projections horizontales après binarisation. Selon la qualité de l'image, le contraste ou la présence de bruit, le seuillage automatique peut produire des résultats imparfaits, ce qui influence directement la précision de l'angle détecté.

Un autre problème observé concerne les **images avec des structures peu marquées**. Dans ces cas, la variance calculée pour les différents angles est faible et proche d'un angle à l'autre, rendant la détection de l'orientation moins fiable. L'algorithme peut alors sélectionner un angle incorrect ou très proche de zéro, même lorsque l'image présente une légère inclinaison.

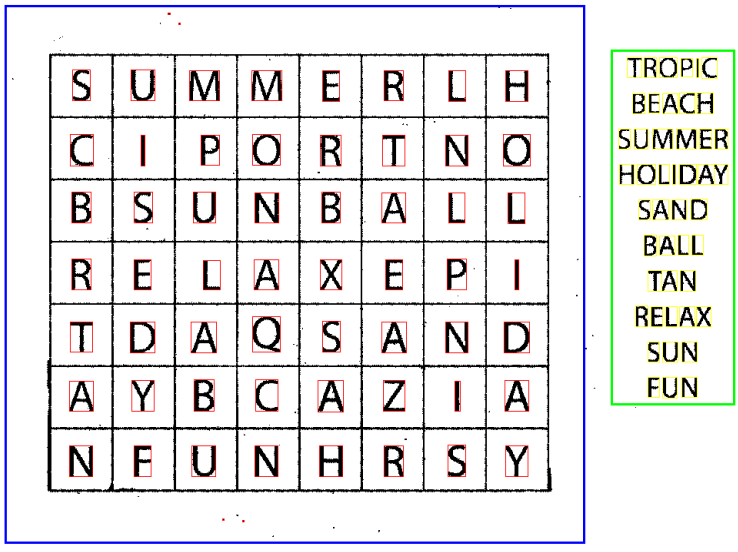


Notamment avec cette image de test où après l'application de la rotation et du prétraitement, celle -ci ne semblait pas subir la rotation automatique. Ce qu'on nous avons réussi à fixer en appliquant le filtre médian avant la rotation et le prétraitement.

Voici l'image avant l'application du fix



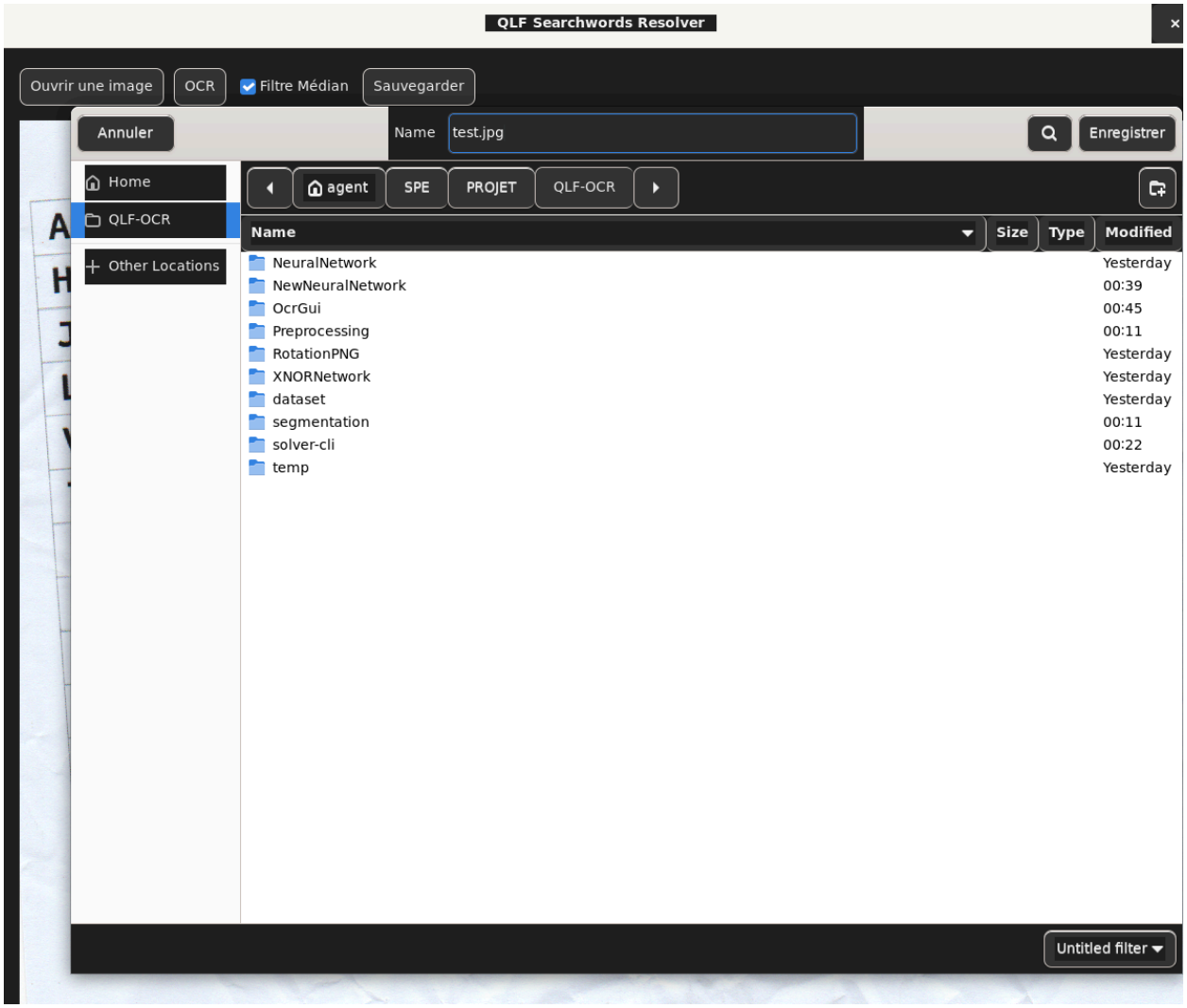
Voici l'image qui a subi le filtre,la rotation,le prétraitement et la détection de grille après le fix.



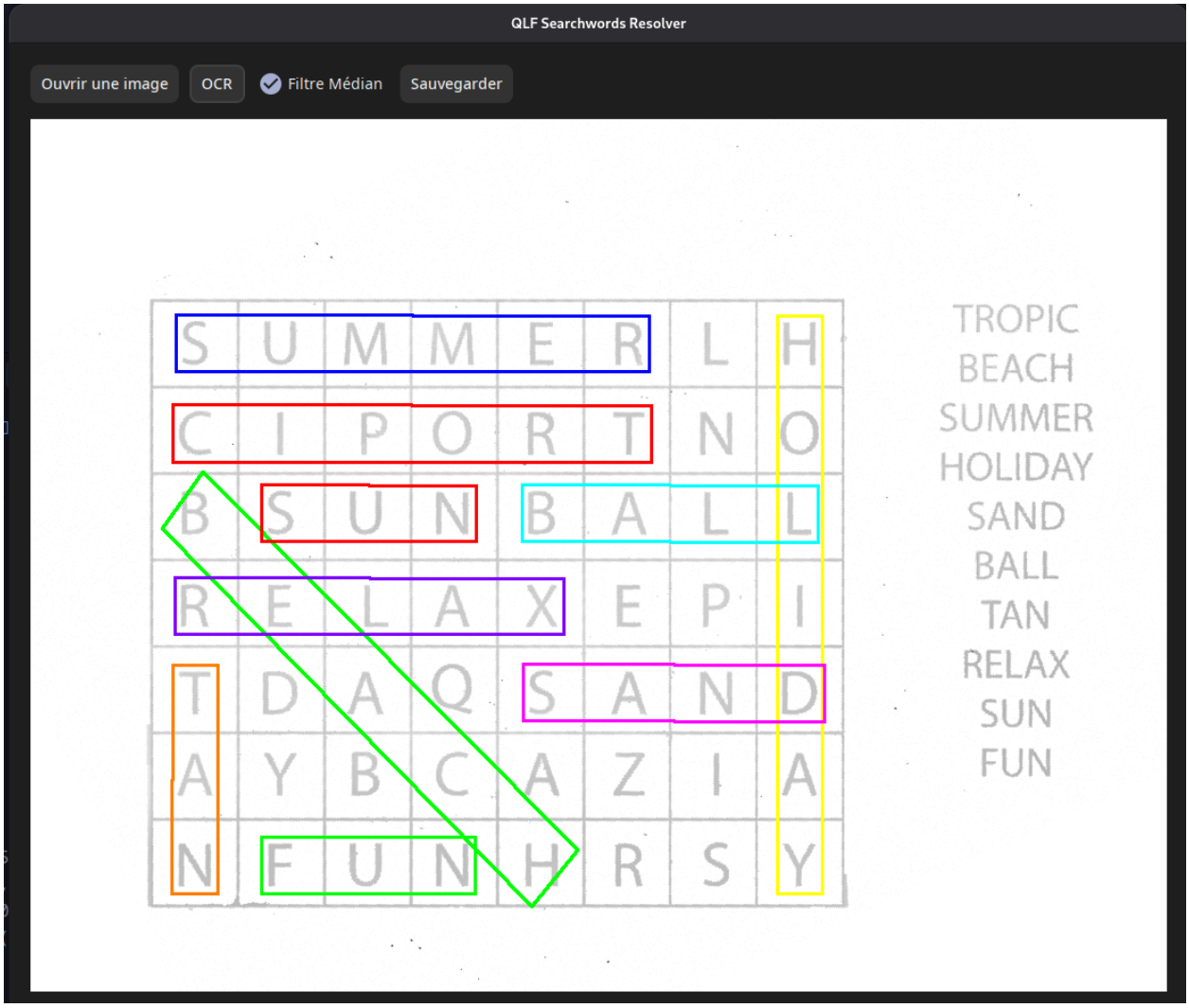
e. Interface (GUI)

L’interface graphique a été conçue avec GTK 3 afin d’offrir une utilisation simple, claire et intuitive du programme. Elle permet à l’utilisateur de charger une image depuis son système, d’afficher un aperçu centralisé et d’appliquer les différents traitements proposés par le projet. La fenêtre principale, intitulée QLF Searchwords Resolver, est dimensionnée par défaut à 1000×700 pixels et centrée à l’écran afin d’améliorer le confort d’utilisation.

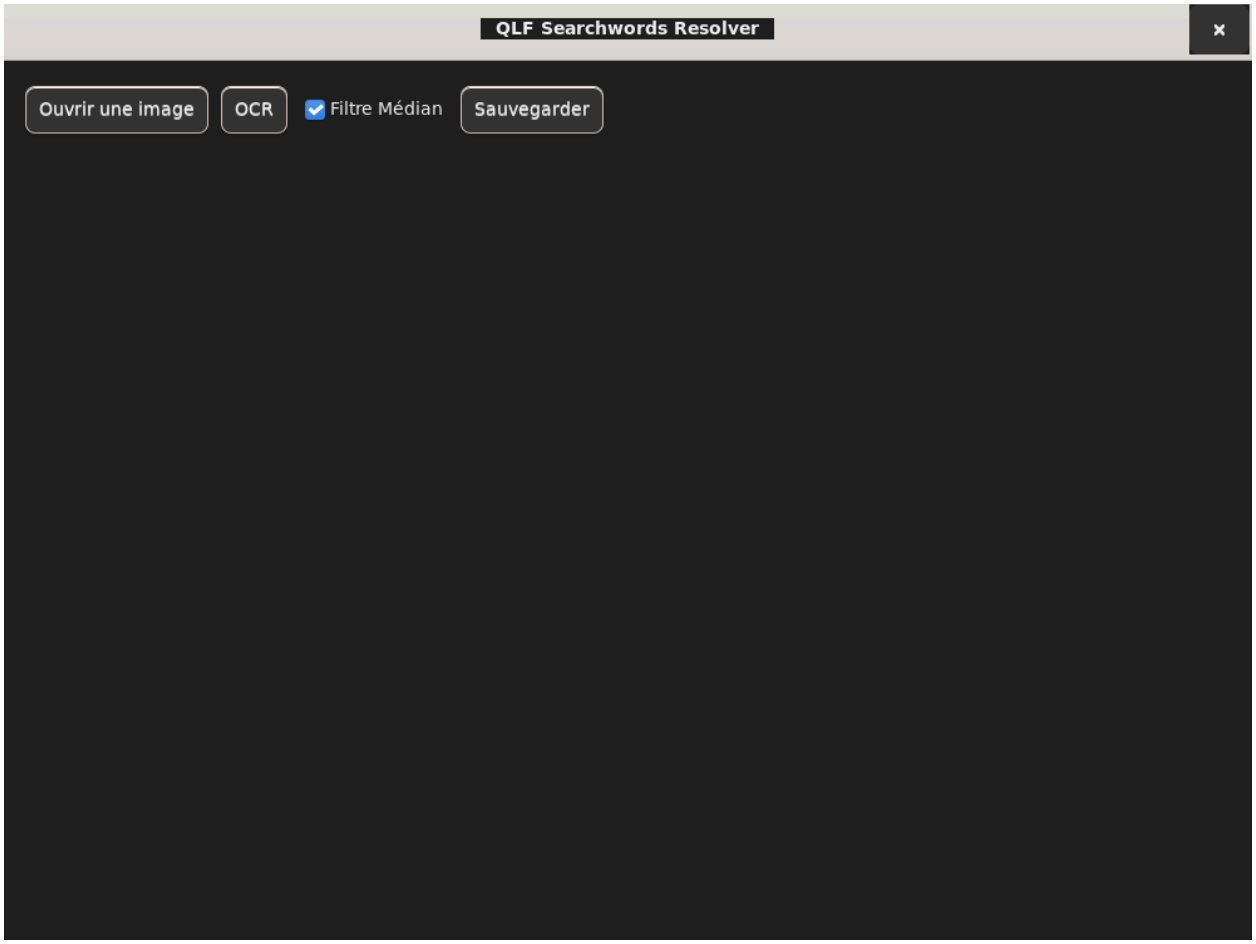
L’interface est organisée de manière verticale. Une barre supérieure regroupe les principaux boutons d’action : Ouvrir une image, OCR, Filtre Médian et Segmentation et Sauvegarder. Le bouton Ouvrir une image permet de sélectionner plusieurs formats d’images courants (PNG, JPG, BMP,etc), qui sont ensuite chargés et affichés dans la zone centrale. Le bouton Sauvegarder offre la possibilité d’exporter l’image affichée après traitement dans différents formats, avec confirmation d’écrasement si nécessaire.



Le bouton OCR permet de lancer un programme externe de reconnaissance de texte en lui passant le chemin de l'image chargée. Une option supplémentaire, représentée par une case à cocher Filtre Médian, permet d'activer ou non un prétraitement avant l'OCR. Une fois le traitement terminé, l'image résultante est automatiquement chargée et affichée dans l'interface.

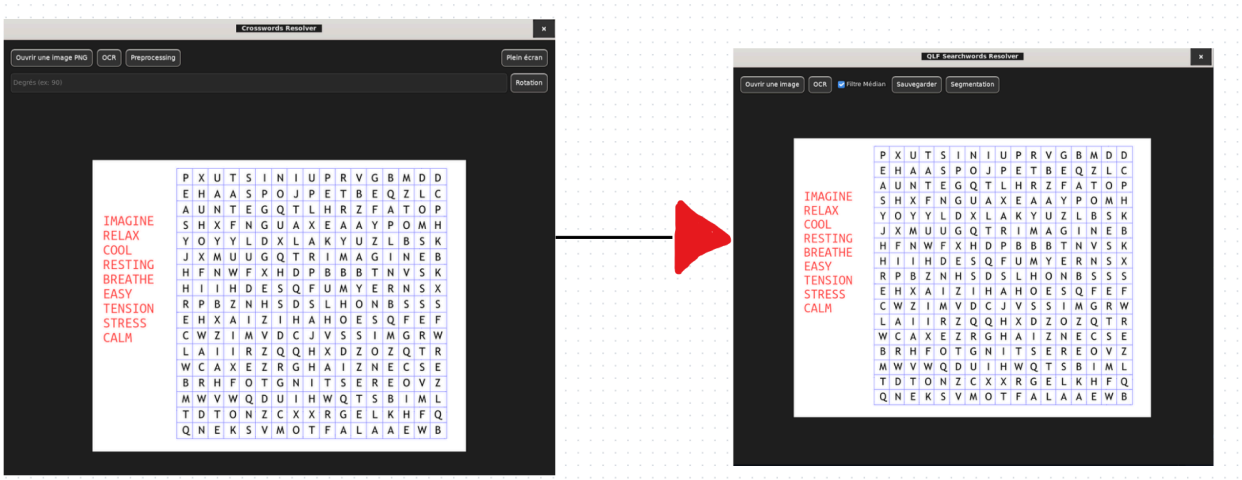


La zone centrale est dédiée à l'aperçu de l'image, avant et après traitement. L'image est centrée horizontalement et verticalement afin d'assurer une bonne lisibilité, quelle que soit sa résolution. Le programme conserve également une copie de l'image originale, ce qui permet d'éviter toute perte de données lors des transformations.



Un thème sombre a été appliqué à l'ensemble de l'interface à l'aide de feuilles de style CSS, améliorant le confort visuel et offrant une apparence moderne et homogène à l'application.

Un problème a toutefois été identifié concernant la fonctionnalité Plein écran, qui provoquait des erreurs d'affichage avec certaines images, notamment en raison de formats et de résolutions incompatibles avec le redimensionnement automatique. Face à ces limitations techniques et afin de garantir la stabilité de l'application, il a été décidé de supprimer la fonctionnalité de plein écran. Cette solution permet d'éviter les dysfonctionnements tout en conservant une interface fiable et fonctionnelle pour les autres traitements.



Nous avons décidé aussi de retirer les autres boutons du types Preprocessing ainsi que Rotation et Degrés pour ne garder que le bouton OCR qui effectue tout en même temps ainsi que Segmentation qui nous montre le découpage de la grille.

f. Pipeline de Traitement Global (Main OCR)

Le fichier **main.c** constitue le point d'entrée et l'orchestrateur de l'application. Il exécute séquentiellement les différentes étapes du traitement, transformant une image brute de mots mêlés en une grille résolue.

1. Prétraitement et Normalisation

Avant toute analyse, l'image subit une série de transformations pour maximiser la qualité des données :

- **Filtrage Médian (Optionnel) :**
  Si activé par l'utilisateur, un filtre médian 3×3 est appliqué pour éliminer le bruit "poivre et sel" sans flouter les contours des lettres.

- **Correction de l'Orientation :**
  L'image est analysée pour détecter son angle d'inclinaison et est automatiquement redressée pour aligner la grille horizontalement.

- **Binarisation et Nettoyage :**
  Une dernière passe de prétraitement (**preprocess_image**) augmente le

contraste et nettoie les artefacts pour faciliter la segmentation.

---

2. Segmentation

L'étape de segmentation décompose l'image en entités logiques :

- Le module **segmentation** analyse l'image pour distinguer la grille de caractères de la liste de mots à chercher.

- Il découpe ensuite chaque zone pour extraire individuellement chaque lettre. Chaque caractère est sauvegardé temporairement sous forme de petite image, et ses coordonnées d'origine sont mémorisées pour la reconstruction finale.

3. Reconnaissance Optique

Le réseau de neurones préalablement entraîné est chargé en mémoire.
Pour chaque lettre segmentée :

- L'image de la lettre est convertie en une matrice binaire 28×28 (**img_to_binary_matrix**).

- Cette matrice alimente le réseau de neurones (**predict**).

- La classe prédite ('A'–'Z') est stockée pour reconstruire virtuellement la grille et la liste des mots en mémoire.

4. Résolution et Visualisation

Une fois les données textuelles extraites :

- **Reconstruction de la Grille :**
  Une matrice 2D virtuelle est assemblée, avec un système de *padding* pour gérer les lignes de longueurs inégales.

- **Appel au Solveur :**
  Pour chaque mot de la liste, le solveur cherche la meilleure correspondance

dans la grille (avec tolérance d'erreur).

- **Rendu Graphique (Draw) :**
  Lorsqu'un mot est trouvé, le programme dessine directement sur l'image originale redressée des cadres colorés autour du mot.

- **Sortie :**
  L'image finale annotée est sauvegardée sous **output/solved.bmp**.

```
--- GRID PREDICTION ---
Row 0: AASSEMBLYOOYHO
Row 1: HDIBOVPDSHTYHB
Row 2: JLLMMTDYLEOPRL
Row 3: LBORSACOPLYCUA
Row 4: VAAMOJOIPYOYSY
Row 5: TSMAIWSMSTLRTA
Row 6: PLMLHYSJSALOAV
Row 7: PHPHIPTRUWBTHA
Row 8: RWMRHTMLTJPYSJ
Row 9: HATLTIDESLHOMA
Row 10: OUSLLBJAOJOYOB
Row 11: OOSOSHMLMHSITT
Row 12: SRTAPYTHONYAAL
Row 13: PNNHELLOHMOOLV

--- WORDS PREDICTION ---
Word 0: HELLO
Word 1: WORLD
Word 2: RUST
Word 3: PHP
Word 4: JAVA
Word 5: ASSEMBLY
Word 6: PYTHON
Word 7: HTML
```

Screenshot de la sortie de ./ocr en ligne de commande avec une image d'exemple.

g. Segmentation, Extraction et Filtrage des Caractères

1. Introduction

Ce rapport présente l'ensemble des mécanismes permettant la segmentation de l'image en deux zones distinctes (Grille et Liste de Mots), ainsi que l'extraction et le filtrage des caractères dans le cadre du projet OCR.
Il décrit également l'ajout récent du **bouton d'activation du filtre médian** accessible dans l'interface utilisateur.

2. Segmentation de l'Image

La segmentation s'appuie sur le fichier `segmentation/image_segmentation.c`.
Elle vise à identifier toutes les composantes pertinentes de l'image, puis à les séparer en zones cohérentes (Grille, Wordlist).

2.a – Détection des Composantes Connexes (BFS)

L'image binaire est parcourue par un algorithme de type **BFS** pour détecter chaque groupe de pixels noirs connecté.

Pour chaque composante :

- position,
- dimensions,
- bounding box

sont enregistrées afin d'être analysées ultérieurement.

```c
typedef struct {
  int min_x, max_x;
  int min_y, max_y;
  int pixel_count;
  bool valid;
} Component;
```

2.b – Clustering via DBSCAN

Les composantes extraites sont regroupées à l'aide de l'algorithme **DBSCAN**, qui se base sur la densité des points et les distances entre centres des composantes.

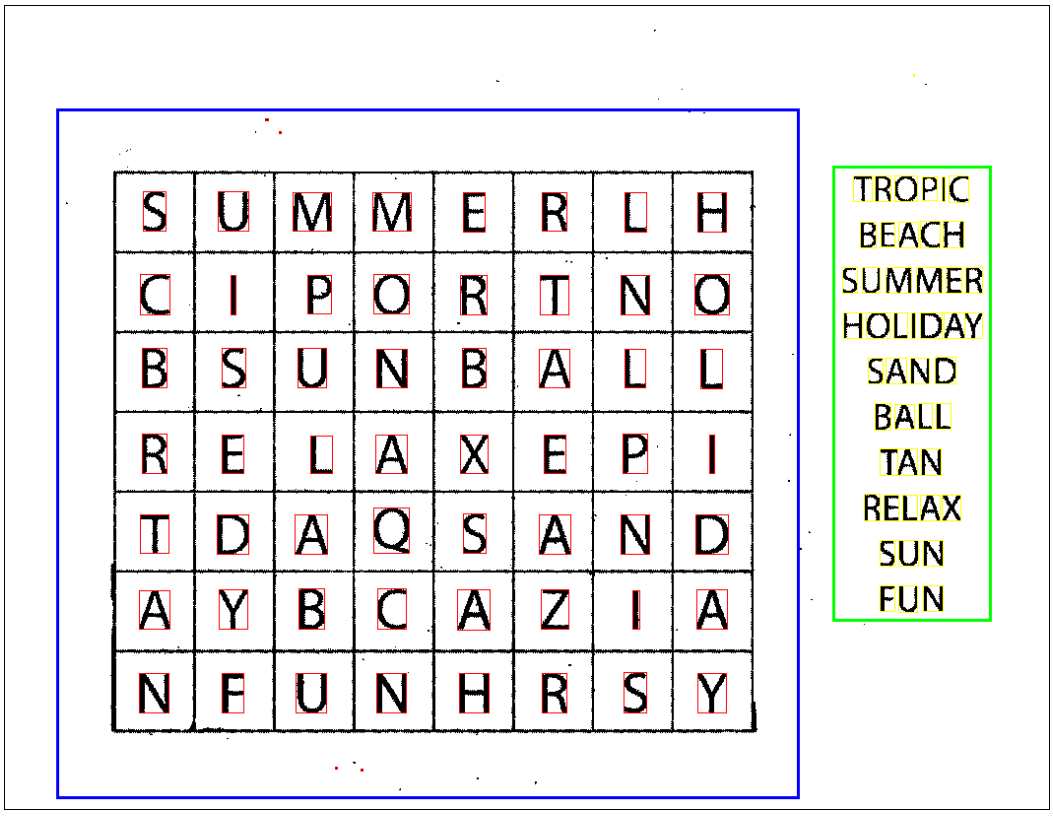
Atouts :

- robuste au bruit,
- pas besoin de connaître le nombre de clusters,
- distingue naturellement les zones compactes (Grille) et les zones étirées (Wordlist).

2.c – Classification des Zones

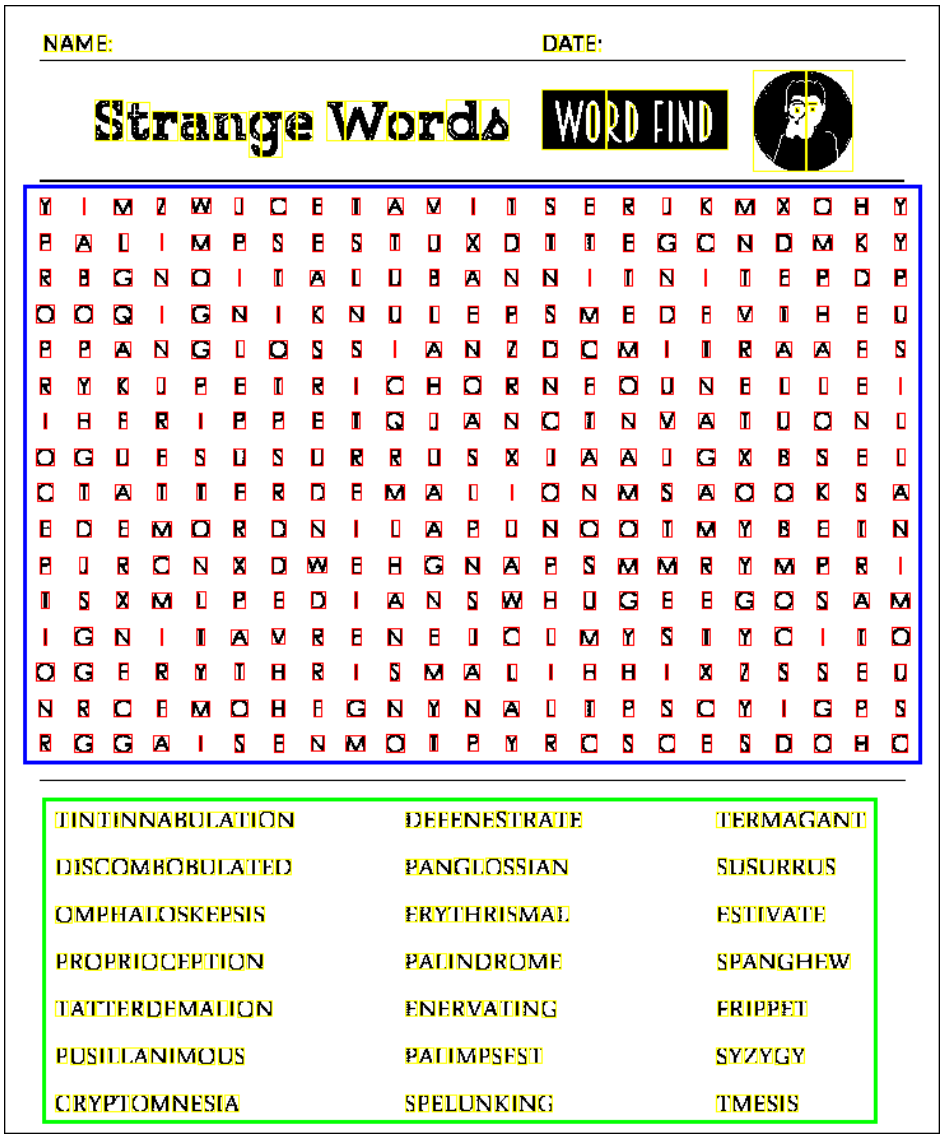
L'analyse des clusters permet de classer chaque zone :

- **Grille** : forte densité, régularité géométrique, alignement vertical / horizontal.
- **Liste de mots** : composantes horizontales plus espacées, situées hors de la grille.
- **UNKNOWN** : cluster ne correspondant à aucun des deux modèles (rare).



Cette image nous a posé plusieurs problèmes principalement en raison du bruit présent dans l'image d'origine. L'objectif était d'utiliser un filtre capable de supprimer ce bruit tout en préservant la qualité des autres images.
Pour atteindre ce résultat, nous avons ajouté un bouton permettant d'activer le filtre uniquement lorsque cela est nécessaire.

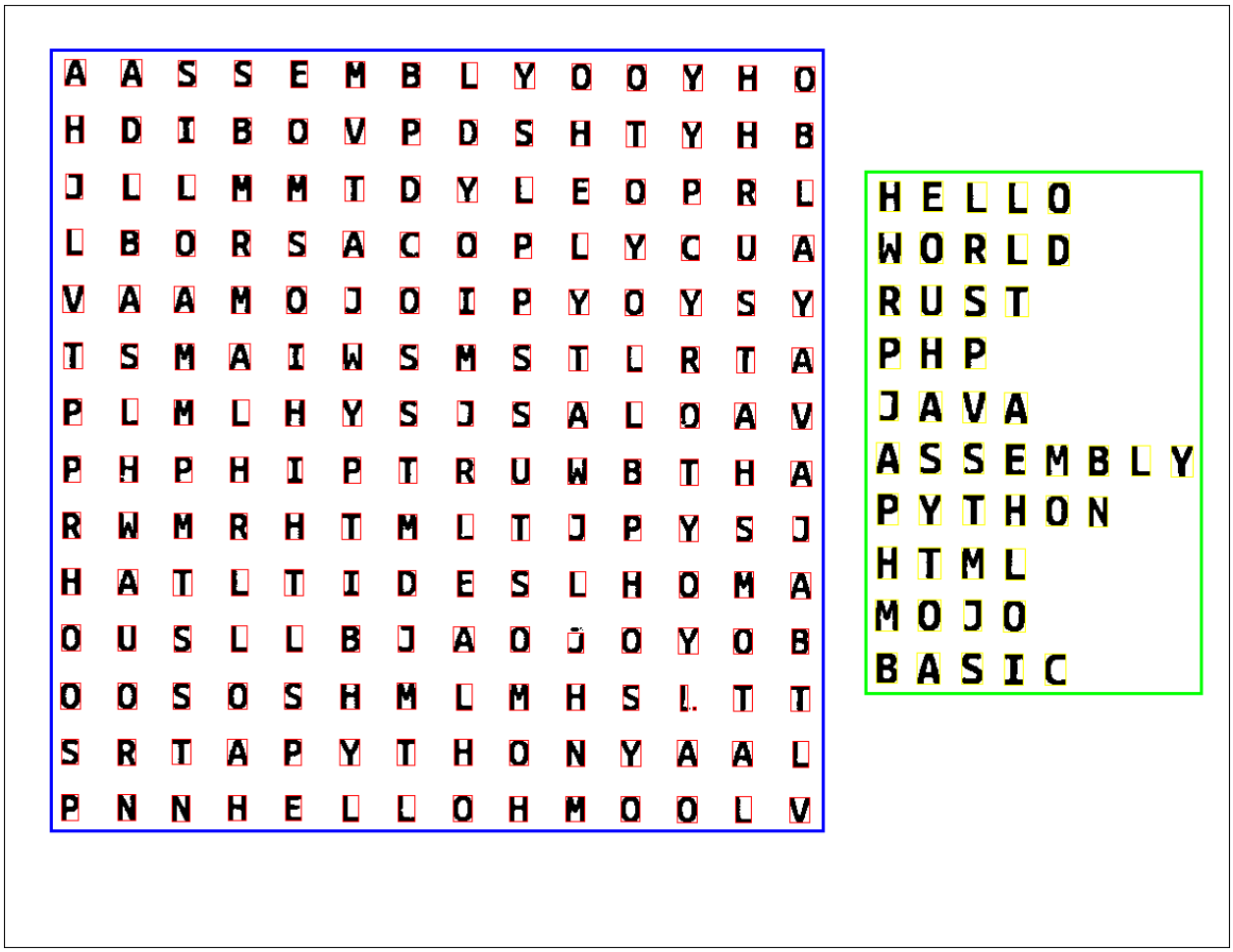




Quant à cette image, le problème provenait du même mécanisme que celui évoqué précédemment. En appliquant la même solution, nous avons pu le résoudre efficacement.
Le filtre médian dégradait un grand nombre de lettres — particulièrement sur cette

image — ce qui rendait la reconnaissance difficile. L'ajout d'un bouton permettant d'activer ou désactiver ce filtre s'est donc révélé être l'une des meilleures solutions.

De plus, lors du parcours horizontal de l'image, le système sélectionnait le titre au lieu de la liste de mots. Pour corriger cela, j'ai ajouté une règle permettant de choisir uniquement le cluster présentant la plus grande taille et une densité suffisante, ce qui garantit l'identification correcte de la liste de mots.



3. Extraction des Lettres

L'extraction et la préparation des caractères se déroulent dans `segmentation/char_segmentation.c`.

3.a – Extraction Initiale et Assignation

Chaque composante est convertie en structure “Character”.
Selon sa position, elle est assignée à :

- `ZONE_GRID`
- ou `ZONE_WORDLIST`.

Cette classification oriente les traitements ultérieurs.

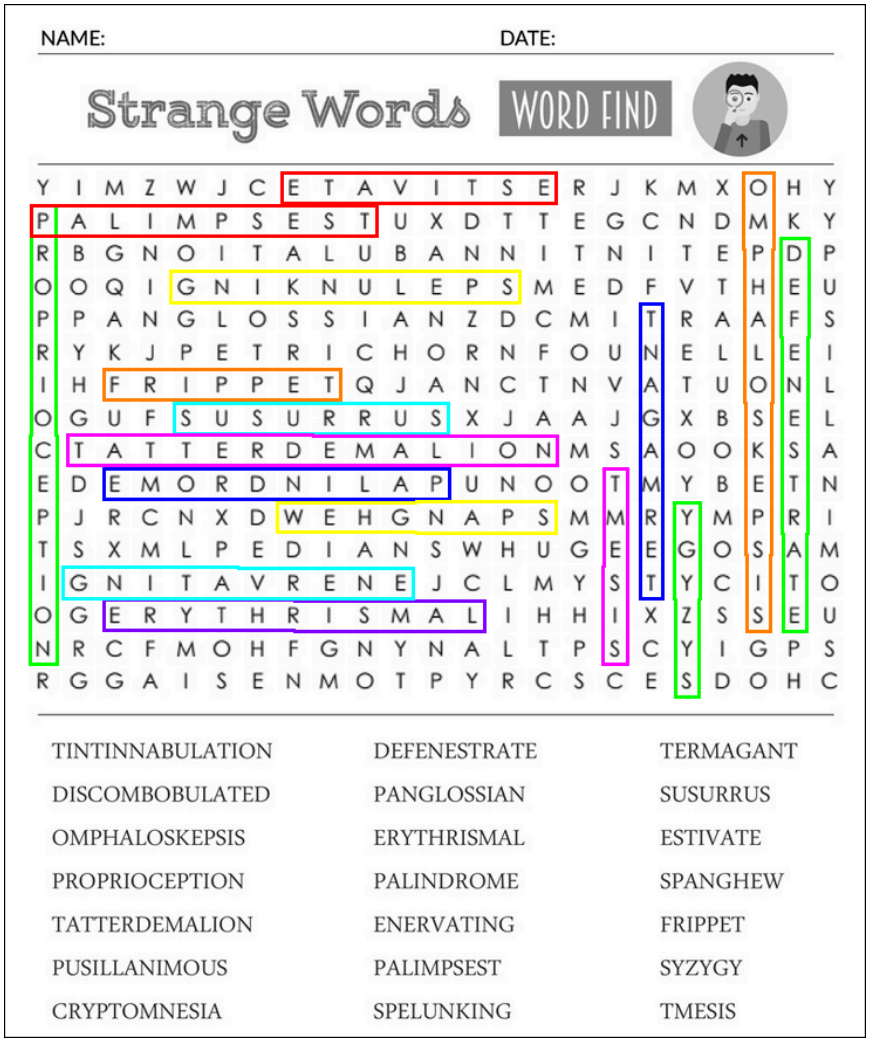
```
typedef enum { ZONE_GRID, ZONE_WORDLIST, ZONE_UNKNOWN } CharacterZone;
```

3.b – Séparation des Caractères Fusionnés

Lorsqu'une composante est trop large (souvent 2× la largeur moyenne), cela suggère deux lettres fusionnées.

L’algorithme :

1. calcule la largeur moyenne des lettres ;
2. détecte les composantes anormalement larges ;
3. construit un histogramme vertical du nombre de pixels noirs ;
4. coupe à l’endroit où l’histogramme présente un minimum (zone de moindre densité).



Nous avons dû ajouter une fonction chargée de vérifier si chaque lettre détectée respecte un ensemble de critères avant d'être acceptée. Cette étape est essentielle pour éviter l'enregistrement d'artefacts, de fragments graphiques ou encore de lettres incorrectement fusionnées.

Concrètement, la fonction analyse plusieurs paramètres, notamment :

- la taille de la composante (largeur, hauteur, surface),
- son ratio largeur/hauteur,
- sa densité en pixels noirs,
- la présence éventuelle de structures anormales indiquant une lettre collée ou mal segmentée.

Si l'un de ces critères n'est pas respecté, la composante est rejetée et n'entre pas dans la liste des lettres valides.

Ce filtrage permet d'assurer une segmentation beaucoup plus propre, particulièrement dans les zones où le bruit ou les déformations provoquent la fusion ou la fragmentation des caractères.

3.c – Filtrage Avancé

Le filtre `filter_letters` élimine automatiquement les composantes indésirables selon :

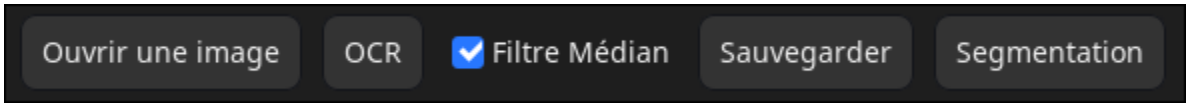
- taille minimale/maximale,
- ratio largeur/hauteur,
- densité interne (pixels noirs vs blancs),
- absence de structures internes incohérentes.

4. Ajout du Bouton "Filtre Médian"

Une nouvelle fonctionnalité a été intégrée dans l'interface afin d'améliorer le comportement de l'OCR sur les images bruitées.

4.a – Interface Utilisateur

Dans `OcrGui/interface.c`, une case à cocher **"Filtre Médian"** a été ajoutée.
Elle est activée par défaut et son état est transmis au programme principal.



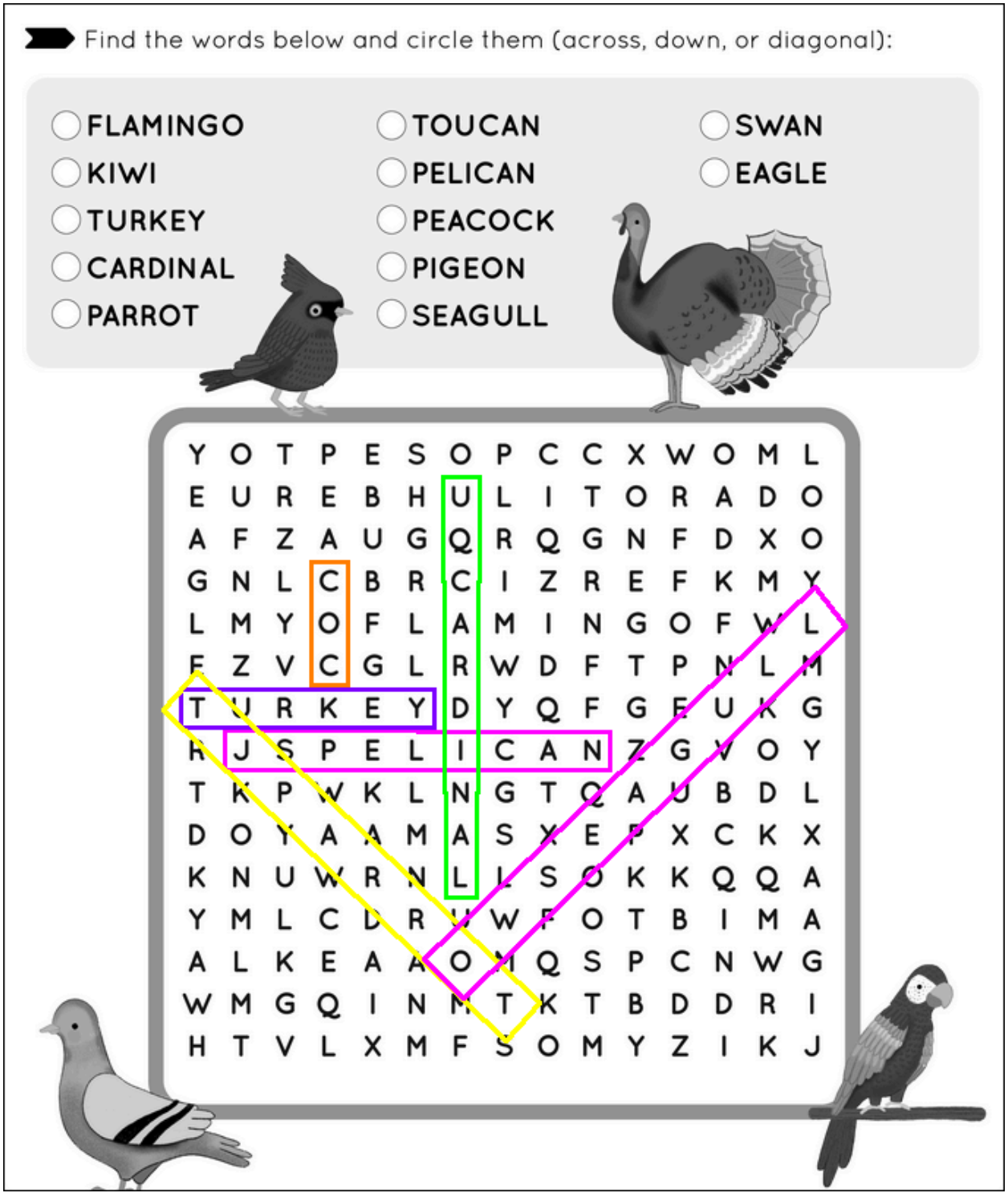
4.b – Traitement dans le Programme

Dans `main.c`, si le paramètre `use_median == 1`, la fonction :

median_filter()

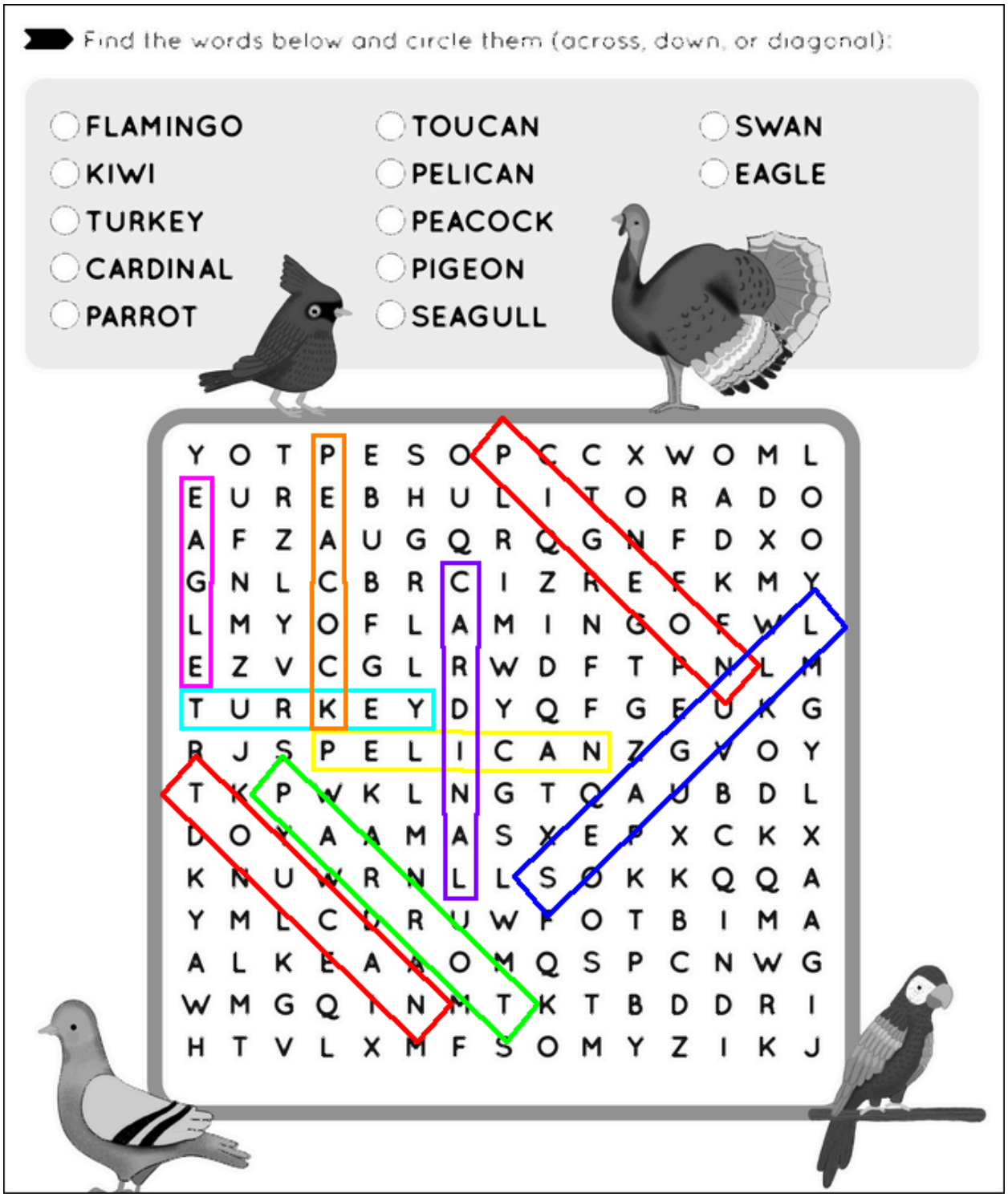
est appliquée **avant** la rotation et la binarisation de l'image.
Le filtre médian 3×3 :

- supprime efficacement le bruit "poivre et sel",
- préserve les contours,
- améliore la qualité de la segmentation.



Voici une image qui n’a pas subi le filtre.

Maintenant la même mais qui a eu le filtre.



On constate qu’il existe une différence frappante : certaines imperfections parviennent encore à passer le filtre de détection des lettres. Grâce au filtre médian, une grande partie de ces défauts disparaît avant même l’analyse.
Cela permet d’obtenir une image beaucoup plus propre, ce qui rend ensuite le travail du solver nettement plus simple et plus fiable.

V. Conclusion.

Tâches	Soutenance 1	Soutenance finale
	Imagerie	
Prétraitement de l'image	50%	100%
Rotation de l'image	50%	100%
Détection de la grille et de la liste de mots	100%	100%
Détection des lettres et des mots	75%	100%
	Réseau de neurones	
Réseau de neurones	60%	100%
	Autres	
Interface Utilisateur	40%	100%
Solveur	100%	100%

Au terme de ce projet OCR, notre équipe a pu explorer et intégrer un ensemble complet de techniques allant du prétraitement d'image à la segmentation, en passant par la rotation corrigée, la détection des caractères et l'élaboration d'un solveur fonctionnel. Cette progression nous a permis de transformer une simple image brute en une grille analysable, propre et interprétable automatiquement.

Chaque membre de l'équipe a contribué à la résolution de problématiques variées : optimisation du réseau de neurones, amélioration de la précision de la rotation, segmentation robuste de la grille et de la liste de mots, filtrage et extraction des caractères, ou encore création d'une interface utilisateur ergonomique. Ces efforts

cumulés ont mené à un système abouti, fiable et capable de traiter des cas d'utilisation variés.

Malgré les défis rencontrés — dépendances logicielles, artefacts visuels, bruit, variations de formats ou encore comportement du plein écran — nous avons su proposer des solutions adaptées, parfois techniques, parfois pragmatiques, mais toujours orientées vers la stabilité et la qualité des résultats.

Ce projet nous a offert une vision concrète de la chaîne complète d'un OCR : de l'image brute à l'interprétation automatique des données textuelles. Il nous a permis de développer des compétences en traitement d'image, apprentissage automatique, développement d'interfaces et algorithmie, tout en travaillant dans une dynamique d'équipe solide.

Le résultat final est à la fois fonctionnel, cohérent et représentatif des connaissances acquises tout au long du semestre. Ce travail constitue une base solide pour de futures améliorations ou extensions possibles dans le domaine de la reconnaissance automatique.

VI. Bibliographie

Comment marche un réseau de neurones ? : **3Blue1Brown sur YouTube**

Livre de Michel Nielsen : **https://neuralnetworksanddeeplearning.com/**

Comment faire une Interface avec GTK3: **aducator sur YouTube**

Les méthodes K-means et DBscan: **wikipédia** (voir leur fonctionnement)